Three-Way Match Engine

Backend Implementation Guide
PO · GRN · Invoice Reconciliation

Stack: Node.js · Express · MongoDB · Gemini API

Version 1.0.0	**Assignment** Backend Developer	**Type** Document Processing Service

Contents

1. Project Overview & Objectives

The Three-Way Match Engine is a backend REST API that automates procurement document validation. It receives uploaded documents (Purchase Orders, Goods Receipt Notes, and Invoices), uses the Gemini API to extract structured JSON data, persists it in MongoDB, and continuously computes a three-way match result to detect discrepancies before payment approval.

> **Core Value:** Automates the manual, error-prone task of cross-checking procurement documents — ensuring you only pay for what was ordered and received.

Objective	Implementation
Parse any procurement document	Gemini 1.5 Flash via inline base64
Store structured data	MongoDB (3 separate collections)
Perform three-way match	matchingService.js — item-level rules
Handle out-of-order uploads	Re-match on every upload event
Return meaningful mismatch reasons	Reason codes with item + quantity context

2. System Architecture

The service follows a synchronous request-response pipeline where parsing and matching are triggered in the same HTTP request. Each upload goes through four stages: Receive → Parse → Store → Match.

Client	HTTP multipart POST with file + documentType
Multer	Validates MIME type, saves file to /uploads, attaches req.file
documentController	Orchestrates: calls geminiService, saves to MongoDB, triggers matching
geminiService	Reads file as base64, sends to Gemini 1.5 Flash with typed prompt, returns parsed JSON
MongoDB	Stores PO/GRN/Invoice in separate collections; MatchResult upserted per poNumber
matchingService	Fetches all docs for poNumber, runs item-level rules, upserts MatchResult
Response	Returns saved document metadata + current match status in one response

3. Tech Stack & Dependencies

Package	Version	Role
express	^4.19.2	HTTP server and routing
mongoose	^8.4.0	MongoDB ODM — schema definition and queries
multer	^1.4.5-lts.1	Multipart file upload middleware
@google/generative-ai	^0.21.0	Gemini API SDK for document parsing
dotenv	^16.4.5	Environment variable management
uuid	^10.0.0	Unique document ID generation
cors	^2.8.5	Cross-Origin Resource Sharing headers
nodemon (dev)	^3.1.3	Auto-restart on file changes in development

---

4. Data Models

Three document collections and one match result collection. All documents share a **poNumber** field as the join key. Documents are stored independently — there is no foreign key constraint so any document can arrive before related ones.

PurchaseOrder

PurchaseOrder Schema

```
{
documentId: String (UUID, unique),
poNumber: String (indexed),
poDate: Date,
vendorName: String,
vendorAddress: String,
buyerName: String,
currency: String,
totalAmount: Number,
items: [{
itemCode: String, // primary match key
sku: String, // secondary match key
description: String, // fallback match key
quantity: Number,
unitPrice: Number,
```

```
unit: String
}],
filePath: String, originalFileName: String, uploadedAt: Date
}
```

GRN

GRN Schema

```
{
documentId: String (UUID, unique),
grnNumber: String,
poNumber: String (indexed),
grnDate: Date,
receivedBy: String,
warehouse: String,
items: [{
itemCode: String,
sku: String,
description: String,
receivedQuantity: Number, // key field for matching
unit: String,
condition: String
}]
}
```

Invoice

Invoice Schema

```
{
documentId: String (UUID, unique),
invoiceNumber: String,
poNumber: String (indexed),
invoiceDate: Date,
vendorName: String,
totalAmount: Number,
taxAmount: Number,
items: [{
itemCode: String,
sku: String,
```

```
description: String,
quantity: Number, // key field for matching
unitPrice: Number,
totalPrice: Number
}]
}
```

MatchResult

MatchResult Schema

```
{
poNumber: String (unique, indexed),
status: "matched" | "partially_matched" | "mismatch" | "insufficient_documents",
po: ObjectId -> PurchaseOrder,
grns: [ObjectId -> GRN],
invoices: [ObjectId -> Invoice],
itemResults: [{
itemKey: String,
description: String,
poQuantity: Number,
grnQuantity: Number,
invoiceQuantity: Number,
status: "matched" | "mismatch",
issues: [String]
}],
reasons: [String],
summary: { totalPOItems, matchedItems, mismatchedItems, totalGRNs, totalInvoices },
lastUpdated: Date
}
```

---

5. Parsing Flow (Gemini Integration)

The **geminiService.js** file handles all Gemini API interactions. It converts uploaded files to base64 inline data and sends them alongside a document-type-specific structured prompt.

Step	Action	File / Function
1	File received via Multer	middleware/upload.js
2	Read file from disk as Buffer	geminiService.js → fs.readFileSync

3	Convert to base64 string	geminiService.js → Buffer.toString("base64")
4	Select prompt by documentType	geminiService.js → getPromptForDocType()
5	Call Gemini 1.5 Flash with inlineData	geminiService.js → model.generateContent()
6	Strip markdown code fences from response	geminiService.js → cleanAndParseJSON()
7	JSON.parse the cleaned string	geminiService.js → cleanAndParseJSON()
8	On parse failure: retry with fix prompt	geminiService.js → fallback block
9	Return { parsed, rawText } to controller	documentController.js → save to MongoDB

> **Supported File Types:** PDF, JPEG, PNG, WEBP, TXT — up to 20MB each. Gemini handles OCR natively for image-based documents.

---

6. Three-Way Matching Logic

All matching logic lives in **src/services/matchingService.js**. The function **performMatching(poNumber)** fetches the PO, all GRNs, and all Invoices for a poNumber, then validates at the item level.

Item Matching Key Strategy

The **normalizeItemKey(item)** function resolves a match key using the following priority:

Priority	Field	Example	Reason
1 (best)	itemCode	ITEM-001	Machine-assigned, authoritative, cross-doc consistent
2	sku	TSC-LAPTOP-15	Product catalog identifier, reliable if present
3 (fallback)	description	laptop 15 inch i7	Normalized to lowercase+trim; best-effort for free-text docs

Validation Rules

Rule	Check	Reason Code
Rule 1	GRN total qty <= PO qty (per item)	grn_qty_exceeds_po_qty
Rule 2	Invoice total qty <= PO qty (per item)	invoice_qty_exceeds_po_qty
Rule 3	Invoice total qty <= GRN total qty (per item)	invoice_qty_exceeds_grn_qty
Rule 4	Invoice date <= PO date	invoice_date_after_po_date
Rule 5	All items in GRN/Invoice exist in PO	item_missing_in_po
Rule 6	Only one PO per poNumber	duplicate_po

Match Status Determination

matched	All items pass all rules and no date violations
partially_matched	Some items match but others have issues
mismatch	All items have at least one rule violation
insufficient_documents	PO, at least one GRN, or at least one Invoice is missing

---

7. Out-of-Order Document Handling

The system stores every document independently in its own collection. There is no enforced arrival order. On every upload, **triggerMatching(poNumber)** is called to recompute and upsert the MatchResult for that poNumber.

Scenario	Intermediate Status	Final Status (after PO arrives)
Invoice first, GRN second, PO last	insufficient_documents (po_not_yet_received)	Recomputed to matched/mismatch once PO arrives
GRN first, PO second, Invoice last	insufficient_documents (no_invoice_received)	Recomputed on Invoice upload
PO first, Invoice before GRN	insufficient_documents (no_grn_received)	Recomputed on GRN upload
Multiple GRNs over time	partially_matched or matched	Each GRN upload re-sums quantities and re-matches

> **Key Design Decision:** The MatchResult is always *upserted* (findOneAndUpdate with upsert:true), not re-created. This means the match record for a given poNumber is a single living document that evolves as more documents arrive.

---

8. API Reference

Method	Path	Description	Parameters
POST	/documents/upload	Upload & parse a document	multipart: file + documentType
GET	/documents/:id	Get parsed document by MongoDB _id	—
GET	/documents	List documents	query: poNumber, type, page, limit
GET	/match/:poNumber	Get three-way match result	—
GET	/match	List all match results	query: status, page, limit
GET	/health	Health check	—

Upload Request Example (curl)

POST /documents/upload

```
curl -X POST http://localhost:3000/documents/upload \
-F "file=@./purchase_order.pdf" \
-F "documentType=po"
```

Upload Response

Response JSON


```
{
"success": true,
"message": "PO document uploaded and parsed successfully.",
"document": {
"id": "665c1a2b3f4e5d6789abcdef",
"documentId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
"documentType": "po",
"poNumber": "PO-2024-0042",
"uploadedAt": "2024-06-02T09:00:00.000Z"
},
"parsedData": { "poNumber": "PO-2024-0042", "items": [...] },
"matchStatus": { "poNumber": "PO-2024-0042", "status": "insufficient_documents" }
}
```

8b. Match Result Response

GET /match/:poNumber — Success Response


```
GET /match/PO-2024-0042

{
"success": true,
"poNumber": "PO-2024-0042",
"matchStatus": "matched",
"reasons": [],
"summary": {
"totalPOItems": 3, "matchedItems": 3, "mismatchedItems": 0,
"totalGRNs": 1, "totalInvoices": 1
},
"itemResults": [
{
"itemKey": "ITEM-001",
"description": "Laptop 15 inch i7 16GB RAM",
```

```
"poQuantity": 10, "grnQuantity": 7, "invoiceQuantity": 7,
"status": "matched", "issues": []
},
...
],
"documents": {
"purchaseOrders": [...], "grns": [...], "invoices": [...]
}
}
```

---

9. File & Folder Structure

Project Structure

```
three-way-match-engine/
■■■ src/
■ ■■■ app.js # Express app, routes, error handler, server start
■ ■■■ config/
■ ■ ■■■ db.js # Mongoose connection setup
■ ■■■ controllers/
■ ■ ■■■ documentController.js # Upload, list, get document endpoints
■ ■ ■■■ matchController.js # Match result endpoints
■ ■■■ middleware/
■ ■ ■■■ upload.js # Multer config: storage, MIME filter, size limit
■ ■■■ models/
■ ■ ■■■ PurchaseOrder.js # PO Mongoose schema
■ ■ ■■■ GRN.js # GRN Mongoose schema
■ ■ ■■■ Invoice.js # Invoice Mongoose schema
■ ■ ■■■ MatchResult.js # Match result Mongoose schema
■ ■■■ routes/
■ ■ ■■■ documentRoutes.js # /documents/* route definitions
■ ■ ■■■ matchRoutes.js # /match/* route definitions
■ ■■■ services/
■ ■■■ geminiService.js # Gemini API integration + prompts
■ ■■■ matchingService.js # Core three-way matching logic
■■■ uploads/ # File storage (gitignored)
■■■ sample-data/ # Example parsed JSON + match results
■■■ postman/ # Postman collection JSON
```

```
■■■ .env.example # Environment variable template
■■■ package.json
■■■ README.md
```

---

10. Configuration & Setup

.env Configuration

```
# .env file
MONGODB_URI=mongodb://localhost:27017/three-way-match
GEMINI_API_KEY=your_gemini_api_key_here
PORT=3000
UPLOAD_DIR=uploads
NODE_ENV=development
```

Local Setup Commands

```
git clone
cd three-way-match-engine
npm install
cp .env.example .env
# Edit .env with your GEMINI_API_KEY and MONGODB_URI
npm run dev
```

---

11. Sample Outputs

Matched Result

```
{ "matchStatus": "matched", "reasons": [],
"summary": { "totalPOItems": 3, "matchedItems": 3, "mismatchedItems": 0 },
"itemResults": [
{ "itemKey":"ITEM-001", "poQuantity":10, "grnQuantity":7,
"invoiceQuantity":7, "status":"matched", "issues":[] },
{ "itemKey":"ITEM-002", "poQuantity":20, "grnQuantity":20,
"invoiceQuantity":20, "status":"matched", "issues":[] }
]
}
```

Mismatch Result

```
{ "matchStatus": "mismatch",
"reasons": [
```

```
"grn_qty_exceeds_po_qty [item: ITEM-001] grn=12 po=10",
"invoice_qty_exceeds_grn_qty [item: ITEM-002] invoice=25 grn=20",
"invoice_date_after_po_date [invoice: INV-2024-007]"
],
"itemResults": [
{ "itemKey":"ITEM-001", "poQuantity":10, "grnQuantity":12,
"invoiceQuantity":10, "status":"mismatch",
"issues":["grn_qty_exceeds_po_qty"] },
{ "itemKey":"ITEM-002", "poQuantity":20, "grnQuantity":20,
"invoiceQuantity":25, "status":"mismatch",
"issues":["invoice_qty_exceeds_po_qty","invoice_qty_exceeds_grn_qty"] }
]
}
```


12. Assumptions & Tradeoffs

Assumption / Tradeoff	Reasoning
One PO per poNumber	Spec requirement; duplicates are flagged as errors via duplicate_po reason
Partial GRN delivery is valid	GRN qty can be less than PO qty without being a mismatch
Gemini for all doc types	Handles PDF, images, and free-text equally; no separate OCR step needed
Synchronous parsing in request	Simpler to implement; tradeoff is higher response latency for large files
No auth layer	Out of scope; would add JWT in production
No retry queue	Failed Gemini parse requires re-upload; a job queue (BullMQ) would fix this
poNumber is the join key	Simpler than a dedicated linking table; works well when PO number is consistent across do

13. Future Improvements

Improvement	Value
Job queue (BullMQ + Redis)	Decouple Gemini parsing from HTTP; retry on timeout
JWT authentication	Role-based access: finance vs warehouse staff
Webhook / event bus	Notify ERP/Slack when match status changes
Unit & integration tests (Jest)	Cover matching rules and API edge cases

Swagger UI at /api-docs	Auto-generated interactive docs
Embedding-based item matching	Vector similarity for description fallback matching
Audit trail	Immutable log of all MatchResult state transitions
Currency normalization	Convert amounts to base currency before comparison
Streaming uploads to cloud	Store files in S3/GCS instead of local disk
Rate limiting + API key management	Protect Gemini quota from abuse

---